Scaler DSML : Business Case Project

Company: Target

Skills required: SQL

Context

Target is one of the world's most recognized brands and one of America's leading retailers. Target makes itself a preferred shopping destination by offering outstanding value, inspiration, innovation and an exceptional guest experience that no other retailer can deliver.

This business case has information of 100k orders from 2016 to 2018 made at Target in Brazil. Its features allows viewing an order from multiple dimensions: from order status, price, payment and freight performance to customer location, product attributes and finally reviews written by customers.

Dataset: https://drive.google.com/drive/folders/1TGEc66YKbD443nslRi1bWgVd238gJCnb

Data is available in 8 csv files:

1. customers.csv
2. geolocation.csv
3. order_items.csv
4. payments.csv
5. reviews.csv
6. orders.csv
7. products.csv
8. sellers.csv

Each feature or columns of different CSV files are described below:

The **customers.csv** contain following features:

Features	Description

customer_id	Id of the consumer who made the purchase.
customer_unique_id	Unique Id of the consumer.
customer_zip_code_prefix	Zip Code of the location of the consumer.
customer_city	Name of the City from where order is made.
customer_state	State Code from where order is made(Ex- sao paulo-SP).

The **sellers.csv** contains following features:

Features	**Description**
seller_id	Unique Id of the seller registered
seller_zip_code_prefix	Zip Code of the location of the seller.
seller_city	Name of the City of the seller.
seller_state	State Code (Ex- sao paulo-SP)

The **order_items.csv** contain following features:

Features	**Description**
order_id	A unique id of order made by the consumers.
order_item_id	A Unique id given to each item ordered in the order.
product_id	A unique id given to each product available on the site.
seller_id	Unique Id of the seller registered in Target.
shipping_limit_date	The date before which shipping of the ordered product must be completed.
price	Actual price of the products ordered .
freight_value	Price rate at which a product is delivered from one point to another.

The **geolocations.csv** contain following features:

Features	**Description**
geolocation_zip_code_prefix	first 5 digits of zip code
geolocation_lat	latitude
geolocation_lng	longitude
geolocation_city	city name
geolocation_state	state

The **payments.csv** contain following features:

Features	Description
order_id	A unique id of order made by the consumers.
payment_sequential	sequences of the payments made in case of EMI.
payment_type	mode of payment used.(Ex-Credit Card)
payment_installments	number of installments in case of EMI purchase.
payment_value	Total amount paid for the purchase order.

The **orders.csv** contain following features:

Features	Description
order_id	A unique id of order made by the consumers.
customer_id	Id of the consumer who made the purchase.
order_status	status of the order made i.e delivered, shipped etc.
order_purchase_timestamp	Timestamp of the purchase.
order_delivered_carrier_date	delivery date at which carrier made the delivery.
order_delivered_customer_date	date at which customer got the product.
order_estimated_delivery_date	estimated delivery date of the products.

The **reviews.csv** contain following features:

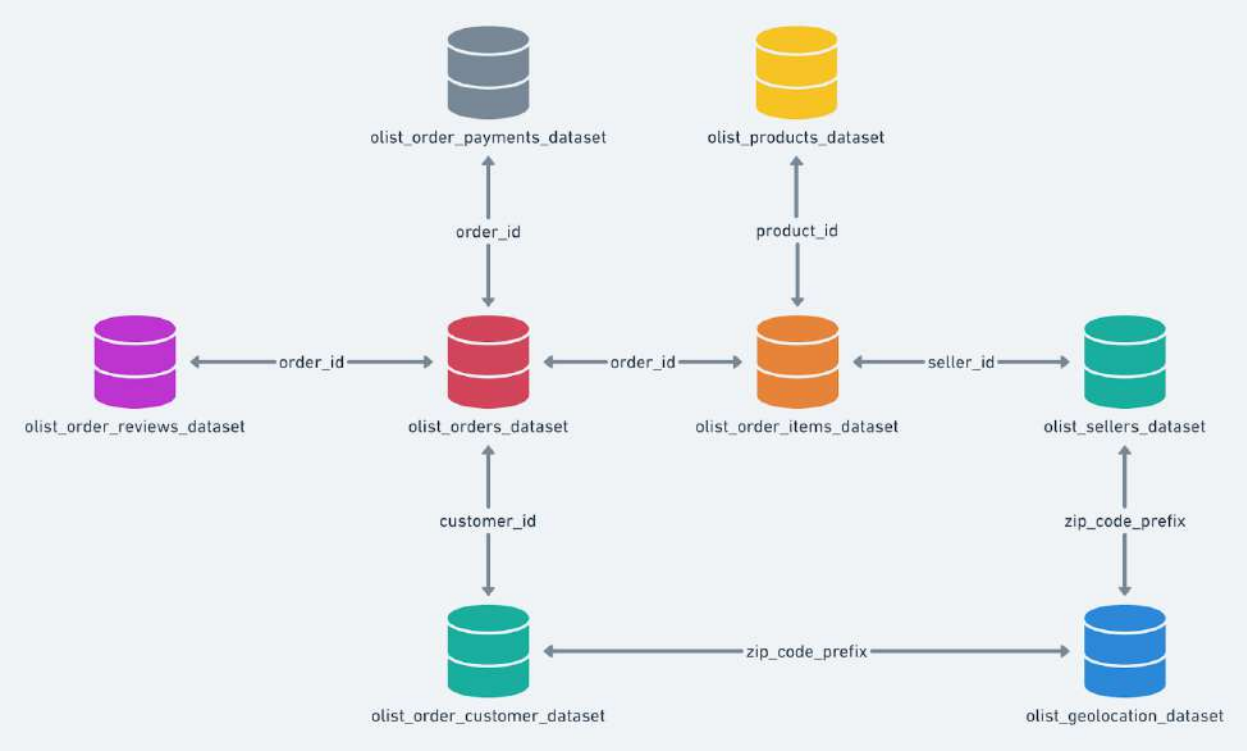
Features	Description
review_id	Id of the review given on the product ordered by the order id.
order_id	A unique id of order made by the consumers.
review_score	review score given by the customer for each order on the scale of 1–5.
review_comment_title	Title of the review
review_comment_message	Review comments posted by the consumer for each order.
review_creation_date	Timestamp of the review when it is created.
review_answer_timestamp	Timestamp of the review answered.

The **products.csv** contain following features:

Features	Description
product_id	A unique identifier for the proposed project.
product_category_name	Name of the product category

product_name_lenght	length of the string which specifies the name given to the products ordered.
product_description_lenght	length of the description written for each product ordered on the site.
product_photos_qty	Number of photos of each product ordered available on the shopping portal.
product_weight_g	Weight of the products ordered in grams.
product_length_cm	Length of the products ordered in centimeters.
product_height_cm	Height of the products ordered in centimeters.
product_width_cm	width of the product ordered in centimeters.

High level overview of relationship between datasets:



Assume you are a data scientist at Target, and are given this data to analyze and provide some insights and recommendations from it.

What does 'good' look like?

1. Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset
    a. Data type of columns in a table

   b. Time period for which the data is given
   c. Cities and States of customers ordered during the given period
2. In-depth Exploration:
   a. Is there a growing trend on e-commerce in Brazil? How can we describe a complete scenario? Can we see some seasonality with peaks at specific months?
   b. What time do Brazilian customers tend to buy (Dawn, Morning, Afternoon or Night)?
3. Evolution of E-commerce orders in the Brazil region:
   a. Get month on month orders by states
   b. Distribution of customers across the states in Brazil
4. Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight and others.
   a. Get % increase in cost of orders from 2017 to 2018 (include months between Jan to Aug only) - You can use “payment_value” column in payments table
   b. Mean & Sum of price and freight value by customer state

5. Analysis on sales, freight and delivery time

1. Calculate days between purchasing, delivering and estimated delivery
2. Find time_to_delivery & diff_estimated_delivery. Formula for the same given below:
   - time_to_delivery = order_purchase_timestamp-order_delivered_customer_date
   - diff_estimated_delivery = order_estimated_delivery_date-order_delivered_customer_date
3. Group data by state, take mean of freight_value, time_to_delivery, diff_estimated_delivery
4. Sort the data to get the following:
5. Top 5 states with highest/lowest average freight value - sort in desc/asc limit 5
6. Top 5 states with highest/lowest average time to delivery
7. Top 5 states where delivery is really fast/ not so fast compared to estimated date

6. Payment type analysis:

1. Month over Month count of orders for different payment types
2. Count of orders based on the no. of payment installments

Evaluation Criteria (80 points)

1. Initial exploration of dataset like checking the characteristics of data (10 points)
2. In-depth Exploration (10 points)
3. Evolution of E-commerce orders in the Brazil region (10 points)
4. Impact on Economy (10 points)
5. Analysis on sales, freight and delivery time (10 points)
6. Payment type analysis (10 points)
7. Actionable Insights (10 points)
8. Recommendations (10 points)

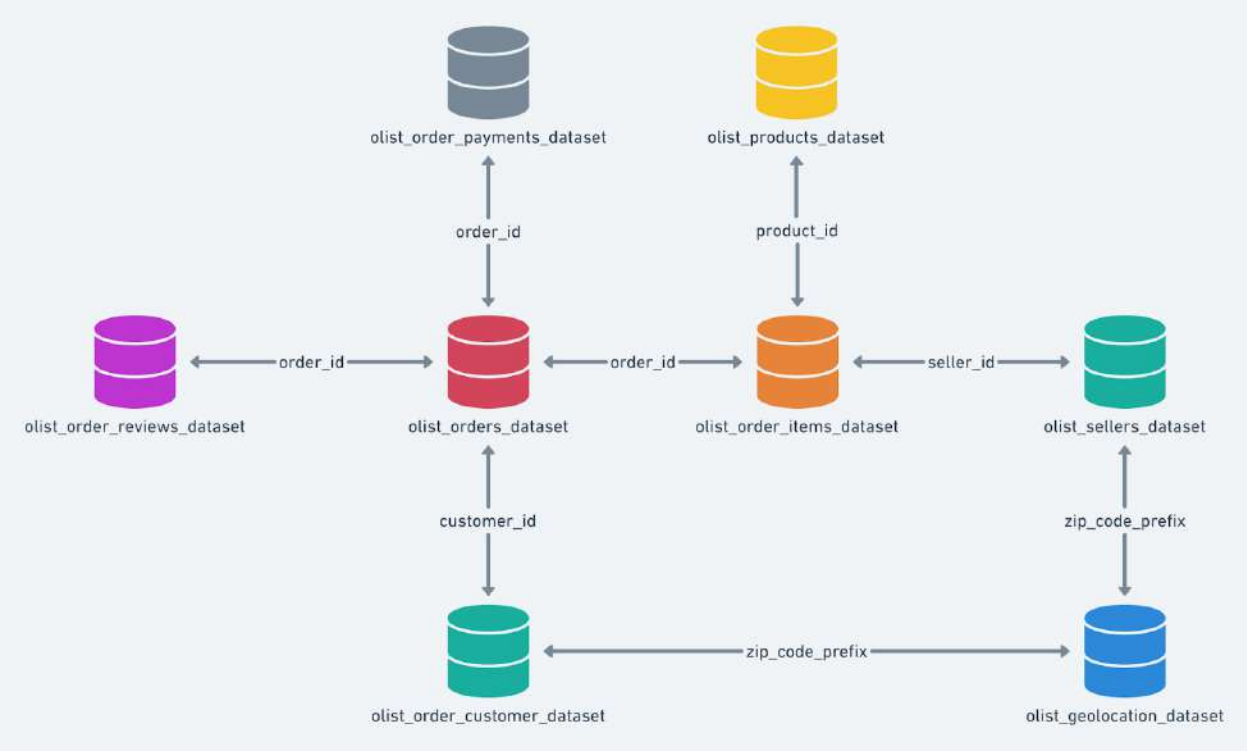
Submission Process <IMP>:

- Use Word doc to paste your SQL queries along with the screenshot of the first 10 rows
- Convert your solutions notebook into PDF, and upload the same on the platform
- Optionally, you may add images/graphs in the text editor by taking screenshots
- After submitting, you will not be allowed to edit your submission

Target dataset schema

Analysis Question #All

Target dataset schema



Path : "C:\Users\ankit\Downloads\Scaler_DSML\classroom\BUSINESS CASE PROJECTS\TARGET - SQL\target_schema.png"

Data is available in 8 csv files:

1. customers.csv

2. geolocation.csv
3. order_items.csv
4. payments.csv
5. reviews.csv
6. orders.csv
7. products.csv
8. sellers.csv

The **customers.csv** contain following features:

Features	Description
customer_id	Id of the consumer who made the purchase.
customer_unique_id	Unique Id of the consumer.
customer_zip_code_prefix	Zip Code of the location of the consumer.
customer_city	Name of the City from where order is made.
customer_state	State Code from where order is made(Ex- sao paulo-SP).

The **sellers.csv** contains following features:

Features	Description
seller_id	Unique Id of the seller registered
seller_zip_code_prefix	Zip Code of the location of the seller.
seller_city	Name of the City of the seller.
seller_state	State Code (Ex- sao paulo-SP)

The **reviews.csv** contain following features:

Features	Description
review_id	Id of the review given on the product ordered by the order id.
order_id	A unique id of order made by the consumers.
review_score	review score given by the customer for each order on the scale of 1–5.

review_comment_title	Title of the review
review_comment_message	Review comments posted by the consumer for each order.
review_creation_date	Timestamp of the review when it is created.
review_answer_timestamp	Timestamp of the review answered.

The **orders.csv** contain following features:

Features	Description
order_id	A unique id of order made by the consumers.
customer_id	Id of the consumer who made the purchase.
order_status	status of the order made i.e delivered, shipped etc.
order_purchase_timestamp	Timestamp of the purchase.
order_delivered_carrier_date	delivery date at which carrier made the delivery.
order_delivered_customer_date	date at which customer got the product.
order_estimated_delivery_date	estimated delivery date of the products.

The **order_items.csv** contain following features:

Features	Description
order_id	A unique id of order made by the consumers.
order_item_id	A Unique id given to each item ordered in the order.
product_id	A unique id given to each product available on the site.
seller_id	Unique Id of the seller registered in Target.
shipping_limit_date	The date before which shipping of the ordered product must be completed.
price	Actual price of the products ordered .
freight_value	Price rate at which a product is delivered from one point to another.

The **payments.csv** contain following features:

Features	Description
order_id	A unique id of order made by the consumers.
payment_sequential	sequences of the payments made in case of EMI.
payment_type	mode of payment used.(Ex-Credit Card)
payment_installments	number of installments in case of EMI purchase.

payment_value	Total amount paid for the purchase order.

The **geolocations.csv** contain following features:

Features	Description
geolocation_zip_code_prefix	first 5 digits of zip code
geolocation_lat	latitude
geolocation_lng	longitude
geolocation_city	city name
geolocation_state	state

The **products.csv** contain following features:

Features	Description
product_id	A unique identifier for the proposed project.
product_category_name	Name of the product category
product_name_lenght	length of the string which specifies the name given to the products ordered.
product_description_lenght	length of the description written for each product ordered on the site.
product_photos_qty	Number of photos of each product ordered available on the shopping portal.
product_weight_g	Weight of the products ordered in grams.
product_length_cm	Length of the products ordered in centimeters.
product_height_cm	Height of the products ordered in centimeters.
product_width_cm	width of the product ordered in centimeters.

Analysis Question #All

1. Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset
    a. Data type of columns in a table
    b. Time period for which the data is given
    c. Cities and States of customers ordered during the given period
2. In-depth Exploration:
    a. Is there a growing trend on e-commerce in Brazil? How can we describe a complete scenario? Can we see some seasonality with peaks at specific months?
    b. What time do Brazilian customers tend to buy (Dawn, Morning, Afternoon or Night)?
3. Evolution of E-commerce orders in the Brazil region:
    a. Get month on month orders by states
    b. Distribution of customers across the states in Brazil
4. Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight and others.
    a. Get % increase in cost of orders from 2017 to 2018 (include months between Jan to Aug only) - You can use “payment_value” column in payments table
    b. Mean & Sum of price and freight value by customer state

5. Analysis on sales, freight and delivery time

1. Calculate days between purchasing, delivering and estimated delivery
2. Find time_to_delivery & diff_estimated_delivery. Formula for the same given below:
    - time_to_delivery = order_purchase_timestamp-order_delivered_customer_date

- diff_estimated_delivery = order_estimated_delivery_date-order_delivered_customer_date

3. Group data by state, take mean of freight_value, time_to_delivery, diff_estimated_delivery
4. Sort the data to get the following:
5. Top 5 states with highest/lowest average freight value - sort in desc/asc limit 5
6. Top 5 states with highest/lowest average time to delivery
7. Top 5 states where delivery is really fast/ not so fast compared to estimated date

6. Payment type analysis:

1. Month over Month count of orders for different payment types
2. Count of orders based on the no. of payment installments

Analysis Question #1

1. Import the dataset and do usual exploratory analysis steps like checking the structure & characteristics of the dataset
    a. Data type of columns in a table
    b. Time period for which the data is given
    c. Cities and States of customers ordered during the given period

Answer

a) Data type of columns in a table

customers.csv

```
SELECT
  column_name,
  data_type
FROM
  `target-project-01-382218.target.INFORMATION_SCHEMA.COLUMNS`
WHERE
  table_name = 'customers'
```

```
SELECT
  column_name,
  data_type
FROM
  `target-project-01-382218.target.INFORMATION_SCHEMA.COLUMNS`
WHERE
  table_name = 'customers'
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS

Row	column_name	data_type
1	customer_id	STRING
2	customer_unique_id	STRING
3	customer_zip_code_prefix	INT64
4	customer_city	STRING
5	customer_state	STRING

sellers.csv

```
SELECT
  column_name,
  data_type
FROM
  `target-project-01-382218.target.INFORMATION_SCHEMA.COLUMNS`
WHERE
  table_name = 'sellers'
```

RUN SAVE SHARE SCHEDULE MORE

```
SELECT
  column_name,
  data_type
FROM
  `target-project-01-382218.target.INFORMATION_SCHEMA.COLUMNS`
WHERE
  table_name = 'sellers'
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS

Row	column_name	data_type
1	seller_id	STRING
2	seller_zip_code_prefix	INT64
3	seller_city	STRING
4	seller_state	STRING

order_items.csv

```
SELECT
  column_name,
  data_type
FROM
  `target-project-01-382218.target.INFORMATION_SCHEMA.COLUMNS`
WHERE
  table_name = 'order_reviews'
```

RUN SAVE SHARE SCHEDULE MORE

```
SELECT
  column_name,
  data_type
FROM
  `target-project-01-382218.target.INFORMATION_SCHEMA.COLUMNS`
WHERE
  table_name = 'order_reviews'
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS

Row	column_name	data_type
1	review_id	STRING
2	order_id	STRING
3	review_score	INT64
4	review_comment_title	STRING
5	review_creation_date	TIMESTAMP
6	review_answer_timestamp	TIMESTAMP

geolocations.csv

```
SELECT
  column_name,
  data_type
FROM
  `target-project-01-382218.target.INFORMATION_SCHEMA.COLUMNS`
WHERE
  table_name = 'geolocation'
```

```
SELECT
  column_name,
  data_type
FROM
  `target-project-01-382218.target.INFORMATION_SCHEMA.COLUMNS`
WHERE
  table_name = 'geolocation'
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS

Row	column_name	data_type
1	geolocation_zip_code_prefix	INT64
2	geolocation_lat	FLOAT64
3	geolocation_lng	FLOAT64
4	geolocation_city	STRING
5	geolocation_state	STRING

payments.csv

```
SELECT
  column_name,
  data_type
FROM
  `target-project-01-382218.target.INFORMATION_SCHEMA.COLUMNS`
WHERE
  table_name = 'payments'
```

```
SELECT
  column_name,
  data_type
FROM
  `target-project-01-382218.target.INFORMATION_SCHEMA.COLU
WHERE
  table_name = 'payments'
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETA

Row	column_name	data_type
1	order_id	STRING
2	payment_sequential	INT64
3	payment_type	STRING
4	payment_installments	INT64
5	payment_value	FLOAT64

orders.csv

```
SELECT
  column_name,
  data_type
FROM
  `target-project-01-382218.target.INFORMATION_SCHEMA.COLUMNS`
WHERE
  table_name = 'orders'
```

```sql
SELECT
  column_name,
  data_type
FROM
  `target-project-01-382218.target.INFORMATION_SCHEMA.COLUMN
WHERE
  table_name = 'orders'
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS

Row	column_name	data_type
1	order_id	STRING
2	customer_id	STRING
3	order_status	STRING
4	order_purchase_timestamp	TIMESTAMP
5	order_approved_at	TIMESTAMP
6	order_delivered_carrier_date	TIMESTAMP
7	order_delivered_customer_date	TIMESTAMP
8	order_estimated_delivery_date	TIMESTAMP

order_reviews.csv

```
SELECT
  column_name,
  data_type
FROM
  `target-project-01-382218.target.INFORMATION_SCHEMA.COLU
WHERE
  table_name = 'orders'
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETA

Row	column_name	data_type
1	order_id	STRING
2	customer_id	STRING
3	order_status	STRING
4	order_purchase_timestamp	TIMESTAMP
5	order_approved_at	TIMESTAMP
6	order_delivered_carrier_date	TIMESTAMP
7	order_delivered_customer_date	TIMESTAMP
8	order_estimated_delivery_date	TIMESTAMP

Note: The column **review_comment_message** not found in the table

products.csv

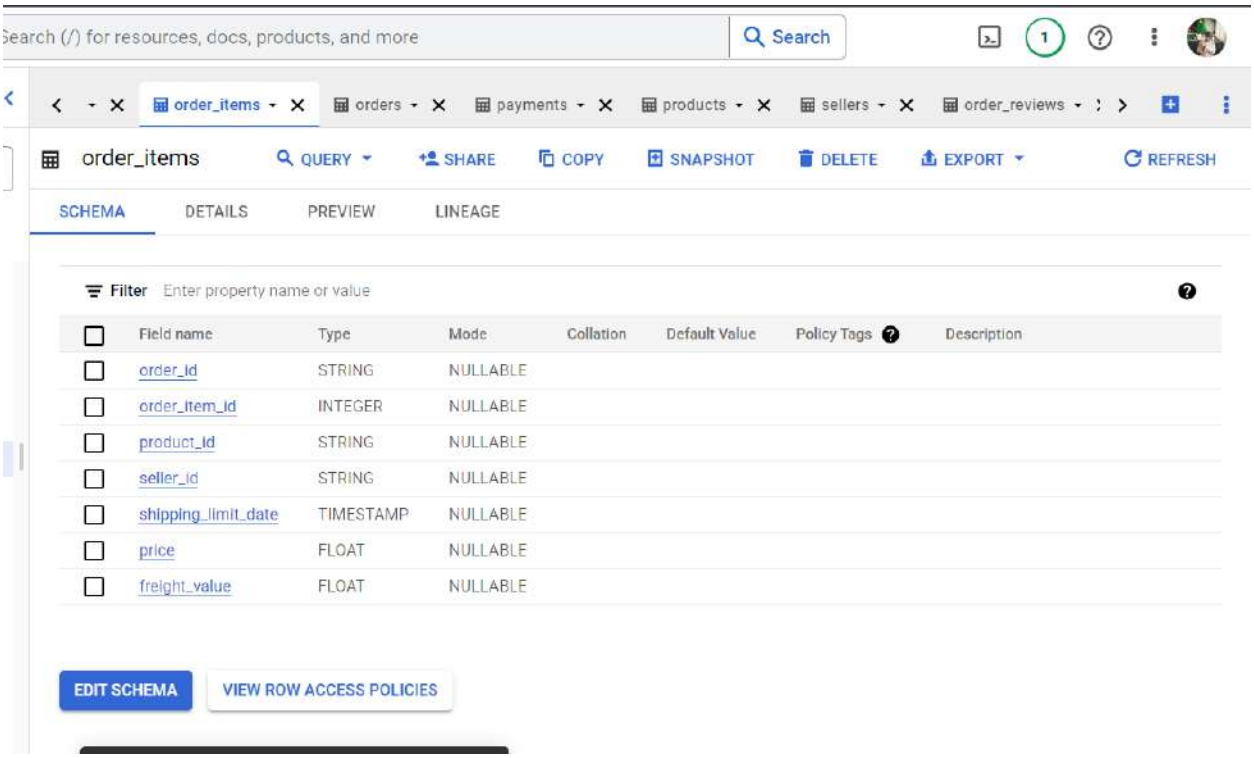
```
SELECT
  column_name,
  data_type
FROM
  `target-project-01-382218.target.INFORMATION_SCHEMA.COLUMNS`
```

```
WHERE
  table_name = 'products'
```

Features	Data Type
product_id	STRING
product_category_name	STRING
product_name_lenght	INTEGER
product_description_lenght	INTEGER
product_photos_qty	INTEGER
product_weight_g	INTEGER
product_length_cm	INTEGER
product_height_cm	INTEGER
product_width_cm	INTEGER

Approach:

SCHEMA tab of each table in **BigQuery** shows the datatype of each Field name in the Table



b) Time period for which the data is given

Approach:

Used **orders** table and found the year range in the **order_purchase_timestamp**

SQL Query:

```
SELECT
  MIN(EXTRACT(Year from order_purchase_timestamp)) AS start_year,
  MAX(EXTRACT(Year from order_purchase_timestamp)) AS end_year
FROM
  `target-project-01-382218.target.orders`
```

*Unsaved query

RUN SAVE SHARE SCHEDULE MORE

```
SELECT
  MIN(EXTRACT(Year from order_purchase_timestamp)) AS start_year,
  MAX(EXTRACT(Year from order_purchase_timestamp)) AS end_year
FROM
  `target-project-01-382218.target.orders`
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS

Row	start_year	end_year
1	2016	2018

Time period of the data is from: **2016 - 2018**

c) Cities and States of customers ordered during the given period

What cities and states the customers who ordered belonged to?

SQL Query:

```
SELECT DISTINCT
c.customer_city
FROM
`target-project-01-382218.target.orders` AS o
JOIN
`target-project-01-382218.target.customers` AS c
ON
o.customer_id = c.customer_id
```

```
1  SELECT DISTINCT
2  c.customer_city
3  FROM
4  `target-project-01-382218.target.orders` AS o
5  JOIN
6  `target-project-01-382218.target.customers` AS c
7  ON
8  o.customer_id = c.customer_id
```

Press

Query results

SAVE RESULTS

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTION GRAPH PREVIEW

Row	customer_city
1	rio de janeiro
2	sao leopoldo
3	general salgado
4	brasilia
5	paranavai
6	cuiaba
7	sao luis
8	maceio
9	hortolandia
10	varzea grande

Results per page: 50 1 – 50 of 4119

The customers orders were from **4119 cities**

```
SELECT DISTINCT
c.customer_state
FROM
`target-project-01-382218.target.orders` AS o
JOIN
`target-project-01-382218.target.customers` AS c
ON
o.customer_id = c.customer_id
```

```
1  SELECT DISTINCT
2  c.customer_state
3  FROM
4  `target-project-01-382218.target.orders` AS o
5  JOIN
6  `target-project-01-382218.target.customers` AS c
7  ON
8  o.customer_id = c.customer_id
```

Press

Query results

SAVE RESULTS

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTION GRAPH PREVIEW

Row	customer_state
1	RJ
2	RS
3	SP
4	DF
5	PR
6	MT
7	MA
8	AL
9	MG
10	PE

Results per page: 50 1 – 27 of 27

The customers orders were from **27 states**

Analysis Question #2

1. In-depth Exploration:
   a. Is there a growing trend on e-commerce in Brazil? How can we describe a complete scenario? Can we see some seasonality with peaks at specific months?
   b. What time do Brazilian customers tend to buy (Dawn, Morning, Afternoon or Night)?

Answer

a) Is there a growing trend on e-commerce in Brazil? How can we describe a complete scenario? Can we see some seasonality with peaks at specific months?

Is there a growing trend on e-commerce in Brazil?

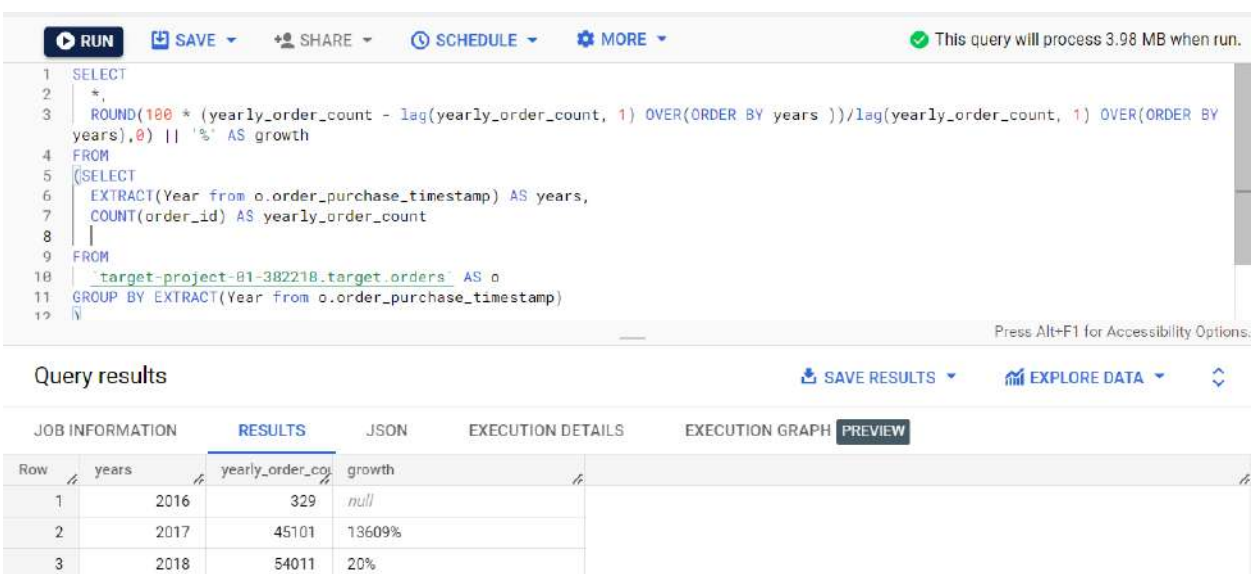
Approach:

Finding the **YEARLY TREND** with count of orders per year

SQL Query:

```
SELECT
  *,
  ROUND(100 * (yearly_order_count - lag(yearly_order_count, 1) OVER(ORDER BY years ))/lag(yearly_order_count, 1) OVER(ORDER BY years),0) ||
FROM
(SELECT
  EXTRACT(Year from o.order_purchase_timestamp) AS years,
  COUNT(order_id) AS yearly_order_count

FROM
  `target-project-01-382218.target.orders` AS o
GROUP BY EXTRACT(Year from o.order_purchase_timestamp)
)
ORDER BY Years
```



There is an exponential growth seen from **2016 to 2017 = 13609%**

Although the growth in the next year **2018** was only = **20%**

How can we describe a complete scenario?

But here the observation maybe biased as the sample set i.e the number of cities and states are not fixed. Means, the huge increase in the rate of growth maybe because there was a massive spread of orders across new cities and states in **2017 and 2018**, which might result in the growth in the number of orders made.

Checking spread in new cities and states

```
SELECT
  *,
  (cities_count - lag(cities_count) OVER(ORDER BY years)) AS new_cities_count,
  (states_count - lag(states_count) OVER(ORDER BY years)) AS new_states_count

FROM
(SELECT
EXTRACT(Year from o.order_purchase_timestamp) AS years,
COUNT (DISTINCT c.customer_city) AS cities_count,
COUNT(DISTINCT c.customer_state) AS states_count

FROM
`target-project-01-382218.target.orders` AS o
JOIN
`target-project-01-382218.target.customers` AS c
ON
o.customer_id = c.customer_id
GROUP BY EXTRACT(Year from o.order_purchase_timestamp)
)

ORDER BY years
```

RUN SAVE SHARE SCHEDULE MORE

```
SELECT
  *,
  (cities_count - lag(cities_count) OVER(ORDER BY years)) AS new_cities_count,
  (states_count - lag(states_count) OVER(ORDER BY years)) AS new_states_count

FROM
(SELECT
EXTRACT(Year from o.order_purchase_timestamp) AS years,
COUNT (DISTINCT c.customer_city) AS cities_count,
COUNT(DISTINCT c.customer_state) AS states_count

FROM
`target-project-01-382218.target.orders` AS o
JOIN
`target-project-01-382218.target.customers` AS c
ON
o.customer_id = c.customer_id
GROUP BY EXTRACT(Year from o.order_purchase_timestamp)
```

Query results SAVE RE

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTION GRAPH PREVI

Row	years	cities_count	states_count	new_cities_count	new_states_count
1	2016	179	21	null	null
2	2017	3290	27	3111	6
3	2018	3277	27	-13	0

As you can see the speculation was somewhat correct :

- Orders received from **3111 new cities and 6 new states** in 2017 from 2016
- The growth in orders were due to huge spread of orders made by customers from new cities and states from **2016 to 2017**
- There is also an observed growth of **20%** from **2017 to 2018** although there has been a reduction in the number of cities from last year

Finding **MONTHLY TREND** with count of orders per month-year

```
SELECT
  *,
  ROUND(100 * (yr_mnt_order_count - lag(yr_mnt_order_count, 1) OVER(PARTITION BY years ORDER BY months ))/lag(yr_mnt_order_count, 1) OVER(P

FROM
(SELECT
  EXTRACT(Year from o.order_purchase_timestamp) AS years,
  EXTRACT(Month from o.order_purchase_timestamp) AS months,
  COUNT(order_id) AS yr_mnt_order_count

FROM
  `target-project-01-382218.target.orders` AS o

GROUP BY
  EXTRACT(Year from o.order_purchase_timestamp),
  EXTRACT(Month from o.order_purchase_timestamp)
)
ORDER BY years, months
```

*Unsaved query | geolocation | *Unsaved query | *Unsaved

RUN | SAVE | SHARE | SCHEDULE | MORE

```
SELECT
  *,
  ROUND(100 * (yr_mnt_order_count - lag(yr_mnt_order_count, 1) OVER(PARTIT
(yr_mnt_order_count, 1) OVER(PARTITION BY years ORDER BY months),0) || '%'

FROM
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTI

Row	years	months	yr_mnt_order_co	growth
1	2016	9	4	null
2	2016	10	324	8000%
3	2016	12	1	-100%
4	2017	1	800	null
5	2017	2	1780	123%
6	2017	3	2682	51%
7	2017	4	2404	-10%
8	2017	5	3700	54%
9	2017	6	3245	-12%
10	2017	7	4026	24%
11	2017	8	4331	8%
12	2017	9	4285	-1%

Can we see some seasonality with peaks at specific months?

- **Peak in orders observed in :**

  1. 2016 - October : 8000%

Row	years	months	yr_mnt_order_co	growth
1	2016	9	4	null
2	2016	10	324	8000%
3	2016	12	1	-100%

  2. 2017 - Februrary, March, May and November

Row	years	months	yr_mnt_order_co	growth
1	2017	1	800	null
2	2017	2	1780	123%
3	2017	3	2682	51%
4	2017	4	2404	-10%
5	2017	5	3700	54%
6	2017	6	3245	-12%
7	2017	7	4026	24%
8	2017	8	4331	8%
9	2017	9	4285	-1%
10	2017	10	4631	8%
11	2017	11	7544	63%
12	2017	12	5673	-25%

The is **no peaks observed** in the orders made for the **Year - 2018**

Row	years	months	yr_mnt_order_co	growth
1	2018	1	7269	null
2	2018	2	6728	-7%
3	2018	3	7211	7%
4	2018	4	6939	-4%
5	2018	5	6873	-1%
6	2018	6	6167	-10%
7	2018	7	6292	2%
8	2018	8	6512	3%
9	2018	9	16	-100%
10	2018	10	4	-75%

There is **no seasonality observed** from this limited time period data

What time do Brazilian customers tend to buy (Dawn, Morning, Afternoon or Night) ?

Assumptions for this analysis

Dawn - 12AM to 7AM

Morning - 8AM to 12PM

Afternoon - 1PM to 6PM

Night - 7PM to 11PM

```
WITH
order_time
AS
```

```
  (SELECT
  order_id,
  order_purchase_timestamp,
  CASE
    WHEN EXTRACT(hour from order_purchase_timestamp) BETWEEN 0 AND 7
    THEN "Dawn"
    WHEN EXTRACT(hour from order_purchase_timestamp) BETWEEN 8 AND 12
    THEN "Morning"
    WHEN EXTRACT(hour from order_purchase_timestamp) BETWEEN 13 AND 18
    THEN "Afternoon"
    ELSE "Night"
  END AS time_of_day
  FROM
  `target-project-01-382218.target.orders`)

SELECT
  time_of_day,
  COUNT(order_id) AS num_of_orders
FROM
  order_time
GROUP BY
  time_of_day
ORDER BY
  num_of_orders DESC
```

RUN SAVE SHARE SCHEDULE

```
1  WITH
2  order_time
3  AS
4    (SELECT
5    order_id,
6    order_purchase_timestamp,
7    CASE
8      WHEN EXTRACT(hour from order_purchase_timestamp) BET
9      THEN "Dawn"
10     WHEN EXTRACT(hour from order_purchase_timestamp) BET
11     THEN "Morning"
12     WHEN EXTRACT(hour from order_purchase_timestamp) BET
13     THEN "Afternoon"
14     ELSE "Night"
15   END AS time_of_day
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DE

Row	time_of_day	num_of_orders
1	Afternoon	38135
2	Night	28331
3	Morning	26502
4	Dawn	6473

- We can see from this analysis that **Afternoon has the highest number of orders purchased**. Brazilian customers have been **more active between 1PM and 6PM,** shortly followed by **Night with the second highest** purchases made between the window of **7PM to 11PM.**
- The **lowest purchases** are made during **Dawn (12AM to 7AM)** most probably due to less customer traffic.
- The top 3 time of the day based on the orders count contribute to **93.49%** to the total orders purchased.
- To increase our sales the stores should capture the active users during these times of the day by more lucrative promotions for their products on their e-commerce sites.

Analysis Question #3

1. Evolution of E-commerce orders in the Brazil region:
    a. Get month on month orders by states
    b. Distribution of customers across the states in Brazil

Get month on month orders by states

```
WITH
state_month_orders
AS
(SELECT
  EXTRACT(Year from o.order_purchase_timestamp) AS years,
  EXTRACT(Month from o.order_purchase_timestamp) AS months,
  c.customer_state,
  COUNT(o.order_id) AS state_orders

FROM
  `target-project-01-382218.target.orders` AS o

JOIN
`target-project-01-382218.target.customers` AS c
ON
o.customer_id = c.customer_id

GROUP BY
  EXTRACT(Year from o.order_purchase_timestamp),
  EXTRACT(Month from o.order_purchase_timestamp),
  c.customer_state
)

SELECT *
FROM
  state_month_orders
ORDER BY
  years,
  months,
  customer_state
```

Row	years	months	customer_state	state_orders
1	2016	9	RR	1
2	2016	9	RS	1
3	2016	9	SP	2
4	2016	10	AL	2
5	2016	10	BA	4
6	2016	10	CE	8
7	2016	10	DF	6
8	2016	10	ES	4
9	2016	10	GO	9
10	2016	10	MA	4
11	2016	10	MG	40

Results per page: 50 1 – 50 of 565

Month and Year with Maximum orders in from which state?

```
WITH
state_month_orders
AS
(SELECT
  EXTRACT(Year from o.order_purchase_timestamp) AS years,
  EXTRACT(Month from o.order_purchase_timestamp) AS months,
  c.customer_state,
  COUNT(o.order_id) AS state_orders

FROM
  `target-project-01-382218.target.orders` AS o

JOIN
`target-project-01-382218.target.customers` AS c
ON
o.customer_id = c.customer_id

GROUP BY
  EXTRACT(Year from o.order_purchase_timestamp),
  EXTRACT(Month from o.order_purchase_timestamp),
  c.customer_state
)

SELECT *
FROM
  state_month_orders
WHERE
  state_orders = (SELECT MAX(state_orders) FROM state_month_orders)
```

Row	years	months	customer_state	state_orders
1	2018	8	SP	3253

With an order count of **3253,** state **SP** has the **highest state orders for 2018-08**

Month and Year with Minimum orders in from which state?

```
WITH
state_month_orders
AS
(SELECT
  EXTRACT(Year from o.order_purchase_timestamp) AS years,
  EXTRACT(Month from o.order_purchase_timestamp) AS months,
  c.customer_state,
  COUNT(o.order_id) AS state_orders

FROM
  `target-project-01-382218.target.orders` AS o

JOIN
`target-project-01-382218.target.customers` AS c
ON
o.customer_id = c.customer_id

GROUP BY
  EXTRACT(Year from o.order_purchase_timestamp),
  EXTRACT(Month from o.order_purchase_timestamp),
  c.customer_state
)

SELECT *
FROM
  state_month_orders
WHERE
  state_orders = (SELECT MIN(state_orders) FROM state_month_orders)
ORDER BY
  years,
  months
```

```
WITH
state_month_orders
AS
(SELECT
  EXTRACT(Year from o.order_purchase_timestamp) AS years,
  EXTRACT(Month from o.order_purchase_timestamp) AS months,
  c.customer_state,
  COUNT(o.order_id) AS state_orders
```

Press

Query results

SAVE RESULTS

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTION GRAPH PREVIEW

Row	years	months	customer_state	state_orders
1	2016	9	RR	1
2	2016	9	RS	1
3	2016	10	RR	1
4	2016	10	PB	1
5	2016	10	PI	1
6	2016	12	PR	1
7	2017	1	MS	1
8	2017	6	AM	1
9	2017	7	RR	1
10	2017	7	AP	1
11	2017	7	TO	1

Results per page: 100 1 – 16 of 16

Minimum order of 1 made by the states in the respective months of the year

Distribution of customers across the states in Brazil

```
SELECT DISTINCT
  customer_state,
  COUNT(customer_id) AS customer_per_state

FROM
  `target-project-01-382218.target.customers`

GROUP BY
  customer_state
ORDER BY
  customer_state
```

Row	customer_state	customer_per_st
1	AC	81
2	AL	413
3	AM	148
4	AP	68
5	BA	3380
6	CE	1336
7	DF	2140
8	ES	2033
9	GO	2020
10	MA	747
11	MG	11635

Results per page: 50 1 – 27 of 27

Which state has the maximum customers from?

```
SELECT DISTINCT
  customer_state,
  COUNT(customer_id) AS customer_per_state

FROM
  `target-project-01-382218.target.customers`

GROUP BY
  customer_state
ORDER BY
  customer_per_state DESC
LIMIT 1
```

```sql
SELECT DISTINCT
  customer_state,
  COUNT(customer_id) AS customer_per_state

FROM
  `target-project-01-382218.target.customers`

GROUP BY
  customer_state
ORDER BY
  customer_per_state DESC
LIMIT 1
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS

Row	customer_state	customer_per_st
1	SP	41746

The maximum customers are from state **SP** with a count of **41746 customers** for the period **2016-2018**

Which state has the lowest customer hold?

```sql
SELECT DISTINCT
  customer_state,
  COUNT(customer_id) AS customer_per_state

FROM
  `target-project-01-382218.target.customers`

GROUP BY
  customer_state
```

```
ORDER BY
  customer_per_state
LIMIT 1
```

```
SELECT DISTINCT
  customer_state,
  COUNT(customer_id) AS customer_per_state

FROM
  `target-project-01-382218.target.customers`

GROUP BY
  customer_state
ORDER BY
  customer_per_state
LIMIT 1
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUT

Row	customer_state	customer_per_st
1	RR	46

The state **RR** has lowest number of customers for the period **2016-2018.**

Analysis Question #4

1. Impact on Economy: Analyze the money movement by e-commerce by looking at order prices, freight and others.
    a. Get % increase in cost of orders from 2017 to 2018 (include months between Jan to Aug only) - You can use “payment_value” column in payments table
    b. Mean & Sum of price and freight value by customer state

Impact on Economy:

Analyze the money movement by e-commerce by looking at order prices, freight and others.

Increase in cost of orders, revenue growth and profitability in 2017 & 2018 (for January to August)

```
WITH
  orders_cost
AS
(SELECT

  EXTRACT(year from o.order_purchase_timestamp) AS year,
  ROUND(SUM(price + freight_value),2) AS cost_of_orders,
  SUM(payment_value) AS amount_paid
FROM
  `target-project-01-382218.target.orders` AS o
LEFT JOIN
  `target-project-01-382218.target.order_items` AS items
ON
  o.order_id = items.order_id
LEFT JOIN
  `target-project-01-382218.target.payments` AS p
ON
  o.order_id = p.order_id
WHERE
  EXTRACT(month from o.order_purchase_timestamp) BETWEEN 1 AND 8
GROUP BY
  year
),

yearly_profit AS
(
SELECT
  *,
  ROUND((amount_paid - cost_of_orders),2) AS pl_per_year,
  ROUND(100*(amount_paid - cost_of_orders) / amount_paid,2) || '%' AS profit_perc
FROM
  orders_cost
ORDER BY year
)

SELECT
  *,

  ROUND(100 * (cost_of_orders - lag(cost_of_orders, 1) OVER(ORDER BY year ))/lag(cost_of_orders, 1) OVER(ORDER BY year),0) || '%' AS cost_i
  ROUND(100 * (ROUND((amount_paid),2) - lag(ROUND((amount_paid),2), 1) OVER(ORDER BY year ))/lag(ROUND((amount_paid),2), 1) OVER(ORDER BY y
FROM
  yearly_profit
ORDER BY
  year
```

RUN SAVE SHARE SCHEDULE MORE

```
  `target-project-01-382218.target.orders` AS o
LEFT JOIN
  `target-project-01-382218.target.order_items` AS items
ON
  o.order_id = items.order_id
LEFT JOIN
  `target-project-01-382218.target.payments` AS p
ON
  o.order_id = p.order_id
WHERE
  EXTRACT(month from o.order_purchase_timestamp) BETWEEN 1 AND 8
GROUP BY
  year
),

yearly_profit AS
(
SELECT
  *,
  ROUND((amount_paid - cost_of_orders),2) AS pl_per_year,
  ROUND(100*(amount_paid - cost_of_orders) / amount_paid,2) || '%' AS profit_perc
FROM
```

Query results

SAVE RESULTS

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTION GRAPH PREVIEW

Row	year	cost_of_orders	amount_paid	pl_per_year	profit_perc	cost_increase	revenue_growth
1	2017	3839687.15	4554984.73...	715297.59	15.7%	null	null
2	2018	8982911.87	11122556.9...	2139645.04	19.24%	134%	144%

- In the data we do observe a significant increase in the cost of orders for 2018, **134% increase over 2017.**
- But such increase in cost has not negatively impacted the overall performance of the store since it is still profitable at **19.24%** which is a **3.54% increase over last years' 15.7%**
- The reason is the high growth in revenue, **about 144% over last year** which has not only helped to reduce the impact from the huge spike in the cost but has also generated profits for the store.

Total and Average cost per Customer state

```
SELECT
  c.customer_state,
  ROUND(AVG(price),2) AS mean_price_per_state,
  ROUND(AVG(freight_value),2) AS mean_freight_per_state,
  ROUND(SUM(price),2) AS total_price_per_state,
  ROUND(SUM(freight_value),2 )AS total_freight_per_state

FROM
`target-project-01-382218.target.customers` AS c
JOIN
`target-project-01-382218.target.orders` AS o
ON
  c.customer_id = o.customer_id
LEFT JOIN
  `target-project-01-382218.target.order_items` AS items
ON
  o.order_id = items.order_id

GROUP BY c.customer_state
ORDER BY c.customer_state
```

RUN SAVE SHARE SCHEDULE MORE

```
  ROUND(SUM(freight_value),2 )AS total_freight_per_state

FROM
`target-project-01-382218.target.customers` AS c
JOIN
`target-project-01-382218.target.orders` AS o
ON
  c.customer_id = o.customer_id
```

Press

Query results

SAVE RESULTS

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTION GRAPH PREVIEW

Row	customer_state	mean_price_per_state	mean_freight_per_state	total_price_per_state	total_freight_per_state
1	AC	173.73	40.07	15982.95	3686.75
2	AL	180.89	35.84	80314.81	15914.59
3	AM	135.5	33.21	22356.84	5478.89
4	AP	164.32	34.01	13474.3	2788.5
5	BA	134.6	26.36	511349.99	100156.68
6	CE	153.76	32.71	227254.71	48351.59
7	DF	125.77	21.04	302603.94	50625.5
8	ES	121.91	22.06	275037.31	49764.6
9	GO	126.27	22.77	294591.95	53114.98
10	MA	145.2	38.26	119648.22	31523.77
11	MG	120.75	20.63	1585308.03	270853.46

Results per page: 50 1 – 27 of 27

Analysis Question #5

5. Analysis on sales, freight and delivery time

1. Calculate days between purchasing, delivering and estimated delivery
2. Find time_to_delivery & diff_estimated_delivery. Formula for the same given below:
   - time_to_delivery = order_purchase_timestamp-order_delivered_customer_date
   - diff_estimated_delivery = order_estimated_delivery_date-order_delivered_customer_date
3. Group data by state, take mean of freight_value, time_to_delivery, diff_estimated_delivery

1. Sort the data to get the following:
2. Top 5 states with highest/lowest average freight value - sort in desc/asc limit 5
3. Top 5 states with highest/lowest average time to delivery
4. Top 5 states where delivery is really fast/ not so fast compared to estimated date

Analysis on sales, freight and delivery time

Calculate days between purchasing, delivering and estimated delivery

Order statuses and timestamp relation

[✔️: date information present ✖️: date information not present]

Order status	order purchase date	order approved date	carrier date	customer delivery date	estimated delivery date
created	✔️	✖️	✖️	✖️	✔️
approved	✔️	✔️	✖️	✖️	✔️
invoiced	✔️	✔️	✖️	✖️	✔️
processing	✔️	✔️	✖️	✖️	✔️
unavailable	✔️	✔️	✖️	✖️	✔️
shipped	✔️	✔️	✔️	✖️	✔️
delivered	✔️	✔️/✖️	✔️/✖️	✔️/✖️	✔️
canceled	✔️	✔️/✖️	✔️/✖️	✔️/✖️	✔️

For our analysis we will be using those products which have been **successfully delivered to the customer i.e it has a customer delivery date.**

```
SELECT
  order_id,
  order_purchase_timestamp,
  order_delivered_customer_date,
  order_estimated_delivery_date,
  DATE_DIFF(EXTRACT(date from order_delivered_customer_date), EXTRACT(date from order_purchase_timestamp), DAY)  AS time_to_delivery,
  DATE_DIFF(EXTRACT(date from order_estimated_delivery_date), EXTRACT(date from order_delivered_customer_date), DAY)  AS diff_estimated_del
  CASE
    WHEN DATE_DIFF(EXTRACT(date from order_delivered_customer_date), EXTRACT(date from order_estimated_delivery_date), DAY) < 0
    THEN "no delay"
    ELSE "delay"
  END AS delay_in_delivery ,
  CASE
```

```
    WHEN DATE_DIFF(EXTRACT(date from order_estimated_delivery_date), EXTRACT(date from order_delivered_customer_date), DAY) < 0
    THEN "after estimated"
    ELSE "before estimated"
  END AS delivery_before_estimated

FROM
  `target-project-01-382218.target.orders`
WHERE
  (order_status = "delivered" OR
  order_status = "canceled") AND
  order_delivered_customer_date IS NOT NULL
```

```
14      THEN "no delay"
15      ELSE "delay"
16    END AS delay_in_delivery ,
17    CASE
18      WHEN DATE_DIFF(EXTRACT(date from order_estimated_delivery_date), EXTRACT(date from order_delivered_customer_date), DAY) < 0
19      THEN "after estimated"
20      ELSE "before estimated"
21    END AS delivery_before_estimated
22
23  FROM
```

Press Alt+F1 for

Query results

SAVE RESULTS EXPLORE D

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTION GRAPH PREVIEW

Row	order_id	order_purchase_timestamp	order_delivered_customer_date	order_estimated_delivery_date	time_to_delivery	diff_estimated_c	delay_in_delivery
1	770d331c84e5b214bd9dc70a1...	2016-10-07 14:52:30 UTC	2016-10-14 15:07:11 UTC	2016-11-29 00:00:00 UTC	7	46	no delay
2	1950d777989f6a877539f5379...	2018-02-19 19:48:52 UTC	2018-03-21 22:03:51 UTC	2018-03-09 00:00:00 UTC	30	-12	delay
3	dabf2b0e35b423f94618bf965f...	2016-10-09 00:56:52 UTC	2016-10-16 14:36:59 UTC	2016-11-30 00:00:00 UTC	7	45	no delay
4	8beb59392e21af5eb9547ae1a...	2016-10-08 20:17:50 UTC	2016-10-19 18:47:43 UTC	2016-11-30 00:00:00 UTC	11	42	no delay
5	b60b53ad0bb7dacacf2989fe2...	2017-05-10 14:03:27 UTC	2017-05-23 13:12:27 UTC	2017-05-18 00:00:00 UTC	13	-5	delay
6	276e9ec344d3bf029ff83a161c...	2017-04-08 21:20:24 UTC	2017-05-22 14:11:31 UTC	2017-05-18 00:00:00 UTC	44	-4	delay
7	1a0b31f08d0d7e87935b819ed...	2017-04-11 13:50:49 UTC	2017-04-18 08:18:11 UTC	2017-05-18 00:00:00 UTC	7	30	no delay
8	cec8f5f7a13e5ab934a486ec9e...	2017-03-17 15:56:47 UTC	2017-04-07 13:14:56 UTC	2017-05-18 00:00:00 UTC	21	41	no delay
9	54e1a3c2b97fb0809da548a59...	2017-04-11 19:49:45 UTC	2017-05-22 16:18:42 UTC	2017-05-18 00:00:00 UTC	41	-4	delay
10	58527ee4726911bee84a0f42c...	2017-03-20 11:01:17 UTC	2017-03-30 14:04:04 UTC	2017-05-18 00:00:00 UTC	10	49	no delay

Results per page: 50 1 – 50 of 96476

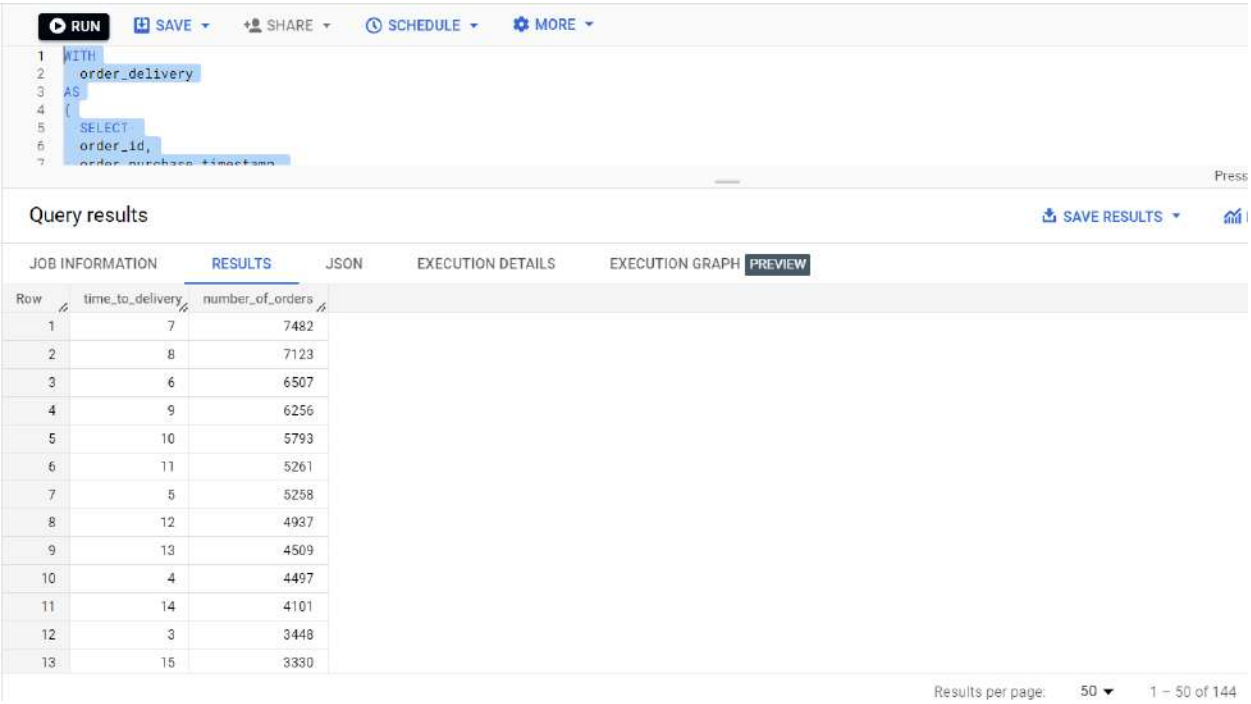
Orders count & delivery time

```
WITH
  order_delivery
AS
(
  SELECT
  order_id,
  order_purchase_timestamp,
  order_delivered_customer_date,
  order_estimated_delivery_date,
  DATE_DIFF(EXTRACT(date from order_delivered_customer_date), EXTRACT(date from order_purchase_timestamp), DAY)  AS time_to_delivery,
  DATE_DIFF(EXTRACT(date from order_estimated_delivery_date), EXTRACT(date from order_delivered_customer_date), DAY)  AS diff_estimated_del
  CASE
    WHEN DATE_DIFF(EXTRACT(date from order_delivered_customer_date), EXTRACT(date from order_estimated_delivery_date), DAY) < 0
    THEN "no delay"
    ELSE "delay"
  END AS delay_in_delivery ,
  CASE
    WHEN DATE_DIFF(EXTRACT(date from order_estimated_delivery_date), EXTRACT(date from order_delivered_customer_date), DAY) < 0
    THEN "after estimated"
    ELSE "before estimated"
  END AS delivery_before_estimated

FROM
  `target-project-01-382218.target.orders`
WHERE
  (order_status = "delivered" OR
  order_status = "canceled") AND
  order_delivered_customer_date IS NOT NULL
ORDER BY
  time_to_delivery
)

SELECT
```

```
  time_to_delivery,
  COUNT(order_id) AS number_of_orders
FROM
  order_delivery
GROUP BY
  time_to_delivery
ORDER BY
  number_of_orders DESC
```



Row	time_to_delivery	number_of_orders
1	7	7482
2	8	7123
3	6	6507
4	9	6256
5	10	5793
6	11	5261
7	5	5258
8	12	4937
9	13	4509
10	4	4497
11	14	4101
12	3	3448
13	15	3330

Most orders between **2016 - 2018** are delivered within **7 days**

<u>Delay - to - On Time(no delay) orders contribution between 2016-2018</u>

```
WITH
  order_delivery
AS
(
  SELECT
  order_id,
  order_purchase_timestamp,
  order_delivered_customer_date,
  order_estimated_delivery_date,
  DATE_DIFF(EXTRACT(date from order_delivered_customer_date), EXTRACT(date from order_purchase_timestamp), DAY)  AS time_to_delivery,
  DATE_DIFF(EXTRACT(date from order_estimated_delivery_date), EXTRACT(date from order_delivered_customer_date), DAY)  AS diff_estimated_del
  CASE
    WHEN DATE_DIFF(EXTRACT(date from order_delivered_customer_date), EXTRACT(date from order_estimated_delivery_date), DAY) < 0
    THEN "no delay"
    ELSE "delay"
  END AS delay_in_delivery ,
  CASE
    WHEN DATE_DIFF(EXTRACT(date from order_estimated_delivery_date), EXTRACT(date from order_delivered_customer_date), DAY) < 0
    THEN "after estimated"
    ELSE "before estimated"
  END AS delivery_before_estimated

FROM
  `target-project-01-382218.target.orders`
WHERE
  (order_status = "delivered" OR
  order_status = "canceled") AND
  order_delivered_customer_date IS NOT NULL
```

```
ORDER BY
  time_to_delivery
),

delay
AS
(
SELECT
  delay_in_delivery,
  COUNT(order_id) AS count_of_orders
FROM
  order_delivery
GROUP BY
  delay_in_delivery
)

SELECT
  *,
  ROUND(100 * count_of_orders / (SELECT SUM(count_of_orders) FROM delay)) || '%' AS percnt_of_orders
FROM
  delay
```

```
1  WITH
2    order_delivery
3  AS
4  (
5    SELECT
6    order_id,
7    order_purchase_timestamp,
8    order_delivered_customer_date,
9    order_estimated_delivery_date,
10   DATE_DIFF(EXTRACT(date from order_delivered_customer_date), EXTRACT(date from order_purchas
11   DATE_DIFF(EXTRACT(date from order_estimated_delivery_date), EXTRACT(date from order_deliver
12   CASE
13     WHEN DATE_DIFF(EXTRACT(date from order_delivered_customer_date), EXTRACT(date from order_
14     THEN "no delay"
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTION GRAPH PREVIEW

Row	delay_in_delivery	count_of_orders	percnt_of_orders
1	no delay	88649	92%
2	delay	7827	8%

Most orders have been delivered within time **about 92% of the total orders that has been delivered successfully to the customers.**

Data by customer state

```
WITH
  order_cust_delivery
AS
(
  SELECT
```

```
  order_id,
  customer_id,
  order_purchase_timestamp,
  order_delivered_customer_date,
  order_estimated_delivery_date,
  DATE_DIFF(EXTRACT(date from order_delivered_customer_date), EXTRACT(date from order_purchase_timestamp), DAY)  AS time_to_delivery,
  DATE_DIFF(EXTRACT(date from order_estimated_delivery_date), EXTRACT(date from order_delivered_customer_date), DAY)  AS diff_estimated_del
  CASE
    WHEN DATE_DIFF(EXTRACT(date from order_delivered_customer_date), EXTRACT(date from order_estimated_delivery_date), DAY) < 0
    THEN "no delay"
    ELSE "delay"
  END AS delay_in_delivery ,
  CASE
    WHEN DATE_DIFF(EXTRACT(date from order_estimated_delivery_date), EXTRACT(date from order_delivered_customer_date), DAY) < 0
    THEN "after estimated"
    ELSE "before estimated"
  END AS delivery_before_estimated

FROM
  `target-project-01-382218.target.orders`
WHERE
  (order_status = "delivered" OR
  order_status = "canceled") AND
  order_delivered_customer_date IS NOT NULL
)

SELECT
  c.customer_state,
  ROUND(AVG(freight_value),2) AS average_freight,
  ROUND(AVG(o.time_to_delivery),2) AS average_delivery_time,
  ROUND(AVG(diff_estimated_delivery),2) AS average_estimated_diff
FROM
  order_cust_delivery AS o
JOIN
  `target-project-01-382218.target.customers` AS c
ON
  o.customer_id = c.customer_id
LEFT JOIN
  `target-project-01-382218.target.order_items` AS items
ON
  o.order_id = items.order_id
GROUP BY
  c.customer_state
```

RUN SAVE SHARE SCHEDULE MORE

```
WITH
  order_cust_delivery
AS
(
  SELECT
  order_id,
  customer_id,
  order_purchase_timestamp,
  order_delivered_customer_date,
  order_estimated_delivery_date,
  DATE_DIFF(EXTRACT(date from order_delivered_customer_date), EXTRACT(date from order_purchase_timestamp), DAY)  AS time_to_delivery,
```

Press

Query results

SAVE RESULTS

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTION GRAPH PREVIEW

Row	customer_state	average_freight	average_delivery_time	average_estimated_diff
1	RJ	20.91	15.07	12.01
2	MG	20.63	11.92	13.34
3	SC	21.51	14.95	11.57
4	SP	15.11	8.66	11.21
5	GO	22.56	15.34	12.29
6	RS	21.61	15.13	14.13
7	BA	26.49	19.19	10.98
8	MT	28.0	17.91	14.57
9	SE	36.57	21.42	10.0
10	PE	32.69	18.22	13.45

Results per page: 50 1 – 27 of 27

Top 5 states with highest average freight value

```
WITH
  order_cust_delivery
AS
(
  SELECT
  order_id,
  customer_id,
  order_purchase_timestamp,
  order_delivered_customer_date,
  order_estimated_delivery_date,
  DATE_DIFF(EXTRACT(date from order_delivered_customer_date), EXTRACT(date from order_purchase_timestamp), DAY)  AS time_to_delivery,
  DATE_DIFF(EXTRACT(date from order_estimated_delivery_date), EXTRACT(date from order_delivered_customer_date), DAY)  AS diff_estimated_del
  CASE
    WHEN DATE_DIFF(EXTRACT(date from order_delivered_customer_date), EXTRACT(date from order_estimated_delivery_date), DAY) < 0
    THEN "no delay"
    ELSE "delay"
  END AS delay_in_delivery ,
  CASE
    WHEN DATE_DIFF(EXTRACT(date from order_estimated_delivery_date), EXTRACT(date from order_delivered_customer_date), DAY) < 0
    THEN "after estimated"
    ELSE "before estimated"
  END AS delivery_before_estimated

FROM
  `target-project-01-382218.target.orders`
WHERE
  (order_status = "delivered" OR
  order_status = "canceled") AND
  order_delivered_customer_date IS NOT NULL
)

SELECT
  c.customer_state,
  ROUND(AVG(freight_value),2) AS average_freight,
  ROUND(AVG(o.time_to_delivery),2) AS average_delivery_time,
  ROUND(AVG(diff_estimated_delivery),2) AS average_estimated_diff
FROM
order_cust_delivery AS o
JOIN
`target-project-01-382218.target.customers` AS c
ON
  o.customer_id = c.customer_id
LEFT JOIN
  `target-project-01-382218.target.order_items` AS items
ON
  o.order_id = items.order_id
GROUP BY
  c.customer_state
ORDER BY
  average_freight DESC
LIMIT 5
```

RUN SAVE SHARE SCHEDULE MORE

```
WITH
  order_cust_delivery
AS
(
  SELECT
  order_id,
  customer_id,
  order_purchase_timestamp,
  order_delivered_customer_date,
  order_estimated_delivery_date,
  DATE_DIFF(EXTRACT(date from order_delivered_customer_date), EXTRACT(date from order_purcha
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTION GRAPH PREVIEW

Row	customer_state	average_freight	average_delivery	average_estimat
1	PB	43.09	20.55	13.04
2	RR	43.09	28.17	18.33
3	RO	41.33	19.66	20.04
4	AC	40.05	20.68	20.98
5	PI	39.12	19.32	11.53

The states are:

- PB
- RR
- RO
- AC
- PI

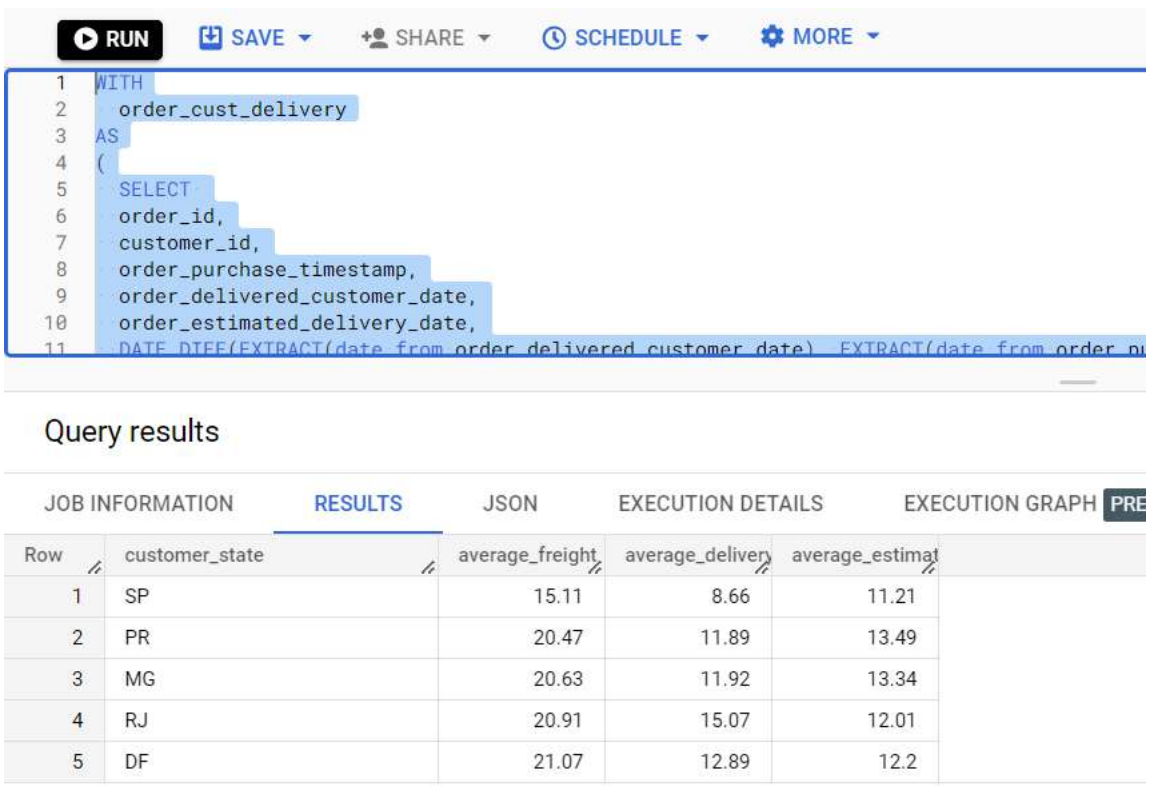
Top 5 states with lowest average freight value

```
WITH
  order_cust_delivery
AS
(
  SELECT
  order_id,
  customer_id,
  order_purchase_timestamp,
  order_delivered_customer_date,
  order_estimated_delivery_date,
  DATE_DIFF(EXTRACT(date from order_delivered_customer_date), EXTRACT(date from order_purchase_timestamp), DAY)  AS time_to_delivery,
  DATE_DIFF(EXTRACT(date from order_estimated_delivery_date), EXTRACT(date from order_delivered_customer_date), DAY)  AS diff_estimated_del
  CASE
    WHEN DATE_DIFF(EXTRACT(date from order_delivered_customer_date), EXTRACT(date from order_estimated_delivery_date), DAY) < 0
    THEN "no delay"
    ELSE "delay"
  END AS delay_in_delivery ,
  CASE
```

```
    WHEN DATE_DIFF(EXTRACT(date from order_estimated_delivery_date), EXTRACT(date from order_delivered_customer_date), DAY) < 0
    THEN "after estimated"
    ELSE "before estimated"
  END AS delivery_before_estimated

FROM
  `target-project-01-382218.target.orders`
WHERE
  (order_status = "delivered" OR
  order_status = "canceled") AND
  order_delivered_customer_date IS NOT NULL
)

SELECT
  c.customer_state,
  ROUND(AVG(freight_value),2) AS average_freight,
  ROUND(AVG(o.time_to_delivery),2) AS average_delivery_time,
  ROUND(AVG(diff_estimated_delivery),2) AS average_estimated_diff
FROM
order_cust_delivery AS o
JOIN
`target-project-01-382218.target.customers` AS c
ON
  o.customer_id = c.customer_id
LEFT JOIN
  `target-project-01-382218.target.order_items` AS items
ON
  o.order_id = items.order_id
GROUP BY
  c.customer_state
ORDER BY
  average_freight
LIMIT 5
```



Row	customer_state	average_freight	average_delivery	average_estimat
1	SP	15.11	8.66	11.21
2	PR	20.47	11.89	13.49
3	MG	20.63	11.92	13.34
4	RJ	20.91	15.07	12.01
5	DF	21.07	12.89	12.2

The states are:

- SP

- PR
- MG
- RJ
- DF

Top 5 states with highest average time to delivery

```
WITH
  order_cust_delivery
AS
(
  SELECT
  order_id,
  customer_id,
  order_purchase_timestamp,
  order_delivered_customer_date,
  order_estimated_delivery_date,
  DATE_DIFF(EXTRACT(date from order_delivered_customer_date), EXTRACT(date from order_purchase_timestamp), DAY)  AS time_to_delivery,
  DATE_DIFF(EXTRACT(date from order_estimated_delivery_date), EXTRACT(date from order_delivered_customer_date), DAY)  AS diff_estimated_del
  CASE
    WHEN DATE_DIFF(EXTRACT(date from order_delivered_customer_date), EXTRACT(date from order_estimated_delivery_date), DAY) < 0
    THEN "no delay"
    ELSE "delay"
  END AS delay_in_delivery ,
  CASE
    WHEN DATE_DIFF(EXTRACT(date from order_estimated_delivery_date), EXTRACT(date from order_delivered_customer_date), DAY) < 0
    THEN "after estimated"
    ELSE "before estimated"
  END AS delivery_before_estimated

FROM
  `target-project-01-382218.target.orders`
WHERE
  (order_status = "delivered" OR
  order_status = "canceled") AND
  order_delivered_customer_date IS NOT NULL
)

SELECT
  c.customer_state,
  ROUND(AVG(freight_value),2) AS average_freight,
  ROUND(AVG(o.time_to_delivery),2) AS average_delivery_time,
  ROUND(AVG(diff_estimated_delivery),2) AS average_estimated_diff
FROM
order_cust_delivery AS o
JOIN
`target-project-01-382218.target.customers` AS c
ON
  o.customer_id = c.customer_id
LEFT JOIN
  `target-project-01-382218.target.order_items` AS items
ON
  o.order_id = items.order_id
GROUP BY
  c.customer_state
ORDER BY
  average_delivery_time DESC
LIMIT 5
```

RUN SAVE SHARE SCHEDULE MORE

```
WITH
  order_cust_delivery
AS
(
  SELECT
  order_id,
  customer_id,
  order_purchase_timestamp,
  order_delivered_customer_date,
  order_estimated_delivery_date,
  DATE_DIFF(EXTRACT(date from order_delivered_customer_date), EXTRACT(date from orde
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTION GRAPH

Row	customer_state	average_freight	average_delivery	average_estimat
1	AP	34.16	28.22	18.4
2	RR	43.09	28.17	18.33
3	AM	33.31	26.34	19.93
4	AL	35.87	24.45	8.74
5	PA	35.63	23.7	14.25

The states are:

- AP
- RR
- AM
- AL
- PA

Top 5 states with lowest average time to delivery

```
WITH
  order_cust_delivery
AS
(
  SELECT
  order_id,
  customer_id,
  order_purchase_timestamp,
  order_delivered_customer_date,
  order_estimated_delivery_date,
  DATE_DIFF(EXTRACT(date from order_delivered_customer_date), EXTRACT(date from order_purchase_timestamp), DAY)  AS time_to_delivery,
  DATE_DIFF(EXTRACT(date from order_estimated_delivery_date), EXTRACT(date from order_delivered_customer_date), DAY)  AS diff_estimated_del
  CASE
    WHEN DATE_DIFF(EXTRACT(date from order_delivered_customer_date), EXTRACT(date from order_estimated_delivery_date), DAY) < 0
    THEN "no delay"
    ELSE "delay"
```

```
  END AS delay_in_delivery ,
  CASE
    WHEN DATE_DIFF(EXTRACT(date from order_estimated_delivery_date), EXTRACT(date from order_delivered_customer_date), DAY) < 0
    THEN "after estimated"
    ELSE "before estimated"
  END AS delivery_before_estimated

FROM
  `target-project-01-382218.target.orders`
WHERE
  (order_status = "delivered" OR
  order_status = "canceled") AND
  order_delivered_customer_date IS NOT NULL
)

SELECT
  c.customer_state,
  ROUND(AVG(freight_value),2) AS average_freight,
  ROUND(AVG(o.time_to_delivery),2) AS average_delivery_time,
  ROUND(AVG(diff_estimated_delivery),2) AS average_estimated_diff
FROM
order_cust_delivery AS o
JOIN
`target-project-01-382218.target.customers` AS c
ON
  o.customer_id = c.customer_id
LEFT JOIN
  `target-project-01-382218.target.order_items` AS items
ON
  o.order_id = items.order_id
GROUP BY
  c.customer_state
ORDER BY
  average_delivery_time
LIMIT 5
```

RUN SAVE SHARE SCHEDULE MORE

```
WITH
  order_cust_delivery
AS
(
  SELECT
  order_id,
  customer_id,
  order_purchase_timestamp,
  order_delivered_customer_date,
  order_estimated_delivery_date,
  DATE_DIFF(EXTRACT(date from order_delivered_customer_date), EXTRACT(date from order_purchase
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTION GRAPH PREVIEW

Row	customer_state	average_freight	average_delivery	average_estimat
1	SP	15.11	8.66	11.21
2	PR	20.47	11.89	13.49
3	MG	20.63	11.92	13.34
4	DF	21.07	12.89	12.2
5	SC	21.51	14.95	11.57

The states are:

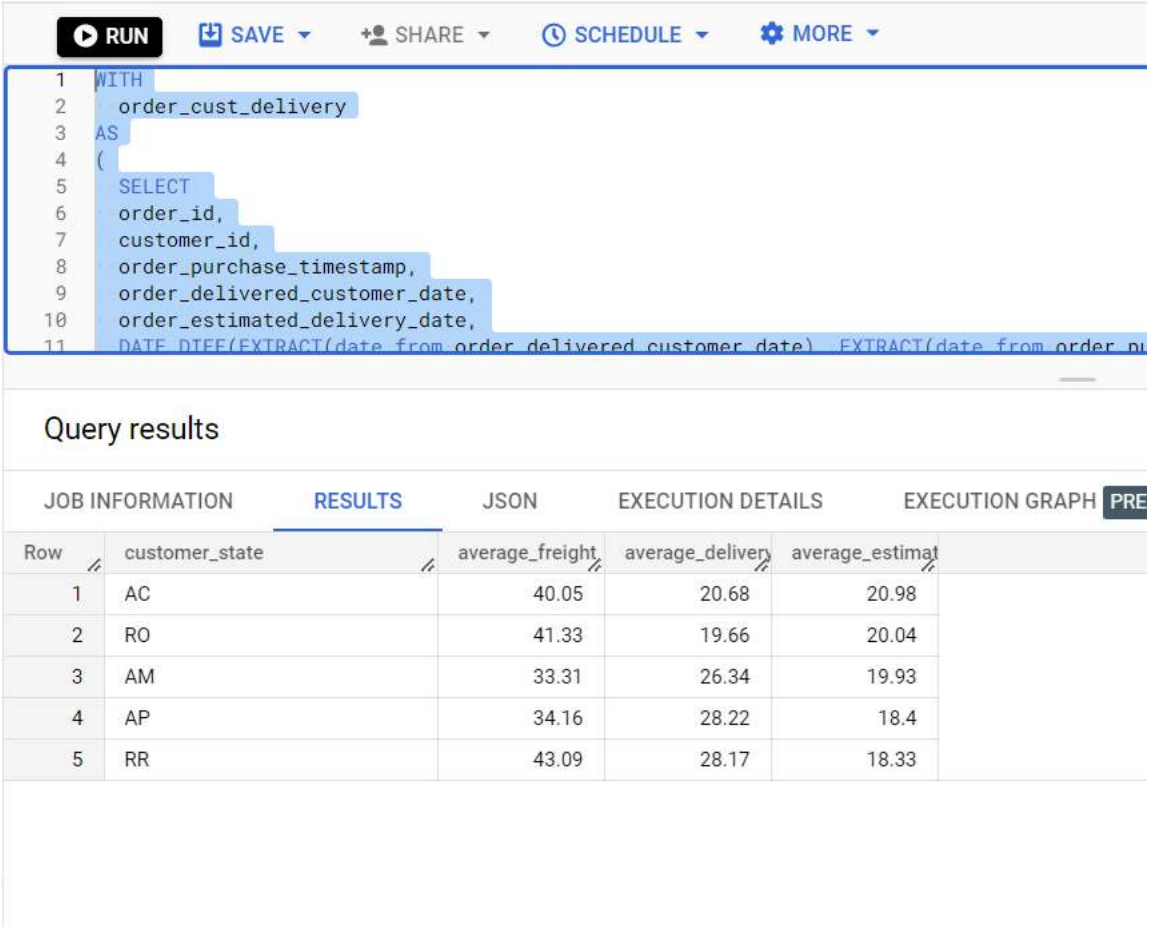
- SP
- PR
- MG
- DF
- SC

Top 5 states with quick delivery before estimated delivery date

```
WITH
  order_cust_delivery
AS
(
  SELECT
  order_id,
  customer_id,
  order_purchase_timestamp,
  order_delivered_customer_date,
  order_estimated_delivery_date,
  DATE_DIFF(EXTRACT(date from order_delivered_customer_date), EXTRACT(date from order_purchase_timestamp), DAY)  AS time_to_delivery,
  DATE_DIFF(EXTRACT(date from order_estimated_delivery_date), EXTRACT(date from order_delivered_customer_date), DAY)  AS diff_estimated_del
  CASE
    WHEN DATE_DIFF(EXTRACT(date from order_delivered_customer_date), EXTRACT(date from order_estimated_delivery_date), DAY) < 0
    THEN "no delay"
    ELSE "delay"
  END AS delay_in_delivery ,
  CASE
    WHEN DATE_DIFF(EXTRACT(date from order_estimated_delivery_date), EXTRACT(date from order_delivered_customer_date), DAY) < 0
    THEN "after estimated"
    ELSE "before estimated"
  END AS delivery_before_estimated

FROM
  `target-project-01-382218.target.orders`
WHERE
  (order_status = "delivered" OR
  order_status = "canceled") AND
  order_delivered_customer_date IS NOT NULL
)

SELECT
  c.customer_state,
  ROUND(AVG(freight_value),2) AS average_freight,
  ROUND(AVG(o.time_to_delivery),2) AS average_delivery_time,
  ROUND(AVG(diff_estimated_delivery),2) AS average_estimated_diff
FROM
order_cust_delivery AS o
JOIN
`target-project-01-382218.target.customers` AS c
ON
  o.customer_id = c.customer_id
LEFT JOIN
  `target-project-01-382218.target.order_items` AS items
ON
  o.order_id = items.order_id
GROUP BY
  c.customer_state
ORDER BY
  average_estimated_diff DESC
LIMIT 5
```



The states are:

- AC
- RO
- AM
- AP
- RR

<u>Top 5 states with late delivery close to estimated delivery date</u>

```
WITH
  order_cust_delivery
AS
(
  SELECT
  order_id,
  customer_id,
  order_purchase_timestamp,
  order_delivered_customer_date,
  order_estimated_delivery_date,
  DATE_DIFF(EXTRACT(date from order_delivered_customer_date), EXTRACT(date from order_purchase_timestamp), DAY)  AS time_to_delivery,
  DATE_DIFF(EXTRACT(date from order_estimated_delivery_date), EXTRACT(date from order_delivered_customer_date), DAY)  AS diff_estimated_del
  CASE
    WHEN DATE_DIFF(EXTRACT(date from order_delivered_customer_date), EXTRACT(date from order_estimated_delivery_date), DAY) < 0
    THEN "no delay"
    ELSE "delay"
  END AS delay_in_delivery ,
```

```
  CASE
    WHEN DATE_DIFF(EXTRACT(date from order_estimated_delivery_date), EXTRACT(date from order_delivered_customer_date), DAY) < 0
    THEN "after estimated"
    ELSE "before estimated"
  END AS delivery_before_estimated

FROM
  `target-project-01-382218.target.orders`
WHERE
  (order_status = "delivered" OR
  order_status = "canceled") AND
  order_delivered_customer_date IS NOT NULL
)

SELECT
  c.customer_state,
  ROUND(AVG(freight_value),2) AS average_freight,
  ROUND(AVG(o.time_to_delivery),2) AS average_delivery_time,
  ROUND(AVG(diff_estimated_delivery),2) AS average_estimated_diff
FROM
order_cust_delivery AS o
JOIN
`target-project-01-382218.target.customers` AS c
ON
  o.customer_id = c.customer_id
LEFT JOIN
  `target-project-01-382218.target.order_items` AS items
ON
  o.order_id = items.order_id
GROUP BY
  c.customer_state
ORDER BY
  average_estimated_diff
LIMIT 5
```

RUN SAVE SHARE SCHEDULE MORE

```
1  WITH
2    order_cust_delivery
3  AS
4  (
5    SELECT
6    order_id,
7    customer_id,
8    order_purchase_timestamp,
9    order_delivered_customer_date,
10   order_estimated_delivery_date,
11   DATE_DIFF(EXTRACT(date from order_delivered_customer_date), EXTRACT(date from order_purchas
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTION GRAPH PREVIEW

Row	customer_state	average_freight	average_delivery	average_estimat
1	AL	35.87	24.45	8.74
2	MA	38.49	21.59	9.91
3	SE	36.57	21.42	10.0
4	ES	22.03	15.59	10.65
5	BA	26.49	19.19	10.98

The states are:

- AL
- MA
- SE
- ES
- BA

Top 5 states across different segments

Position	highest average freight value	lowest average freight value	highest average time to delivery	lowest average time to delivery	quick delivery before estimated delivery date	late delivery close to estimated delivery date
1st	PB	SP	AP	SP	AC	AL
2nd	RR	PR	RR	PR	RO	MA
3rd	RO	MG	AM	MG	AM	SE
4th	AC	RJ	AL	DF	AP	ES
5th	PI	DF	PA	SC	RR	BA

In this table of the the **Top 5 within each category:**

- **AL** has the **fourth highest average time to delivery** and also the **lowest difference to estimated delivery** this there might be large number of delays in order delivery

In-depth analysis for delayed and on-time(no delay) orders for the state AL

```
WITH
  order_cust_delivery
AS
(
  SELECT
  order_id,
  customer_id,
  order_purchase_timestamp,
  order_delivered_customer_date,
  order_estimated_delivery_date,
  DATE_DIFF(EXTRACT(date from order_delivered_customer_date), EXTRACT(date from order_purchase_timestamp), DAY)  AS time_to_delivery,
  DATE_DIFF(EXTRACT(date from order_estimated_delivery_date), EXTRACT(date from order_delivered_customer_date), DAY)  AS diff_estimated_del
  CASE
    WHEN DATE_DIFF(EXTRACT(date from order_delivered_customer_date), EXTRACT(date from order_estimated_delivery_date), DAY) < 0
    THEN "no delay"
    ELSE "delay"
  END AS delay_in_delivery ,
  CASE
    WHEN DATE_DIFF(EXTRACT(date from order_estimated_delivery_date), EXTRACT(date from order_delivered_customer_date), DAY) < 0
    THEN "after estimated"
    ELSE "before estimated"
  END AS delivery_before_estimated

FROM
  `target-project-01-382218.target.orders`
WHERE
  (order_status = "delivered" OR
  order_status = "canceled") AND
  order_delivered_customer_date IS NOT NULL
),
delay_count
AS

(SELECT
  c.customer_state,
  delay_in_delivery,
  COUNT(o.order_id) AS orders_count,

FROM
```

```
order_cust_delivery AS o
JOIN
`target-project-01-382218.target.customers` AS c
ON
  o.customer_id = c.customer_id
LEFT JOIN
  `target-project-01-382218.target.order_items` AS items
ON
  o.order_id = items.order_id
GROUP BY
  c.customer_state, delay_in_delivery
ORDER BY c.customer_state, delay_in_delivery)

(SELECT
*,
ROUND(100*orders_count/SUM(orders_count) OVER(PARTITION BY customer_state),0) AS perc_of_orders
FROM delay_count
ORDER BY delay_in_delivery, perc_of_orders DESC
LIMIT 3)

UNION ALL

(SELECT
*,
ROUND(100*orders_count/SUM(orders_count) OVER(PARTITION BY customer_state),0) AS perc_of_orders
FROM delay_count
ORDER BY delay_in_delivery DESC, perc_of_orders
LIMIT 3)
```

*Unsaved query 4 | *Unsaved query 5 | *Unsaved query 6 | *Unsaved query 7

RUN | SAVE | SHARE | SCHEDULE | MORE

```
58 LIMIT 3)
59
60 UNION ALL
61
62 (SELECT
63 *,
64 ROUND(100*orders_count/SUM(orders_count) OVER(PARTITION BY customer_state),0) AS perc_of_orde
65 FROM delay_count
66 ORDER BY delay_in_delivery DESC, perc_of_orders
67 LIMIT 3)
68
69
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTION GRAPH PREVIEW

Row	customer_state	delay_in_delivery	orders_count	perc_of_orders
1	AL	delay	103	24.0
2	MA	delay	163	20.0
3	SE	delay	61	16.0
4	AL	no delay	324	76.0
5	MA	no delay	637	80.0
6	SE	no delay	314	84.0

- As you can see the state **AL has the highest delayed orders and the lowest orders delivered before estimated delivery date among all states.**
- Maybe we need to implement measures to reduce such delays by changing the delivery carrier service providers delivering in these states.

Analysis Question #6

6. Payment type analysis:

1. Month over Month count of orders for different payment types
2. Count of orders based on the no. of payment installments

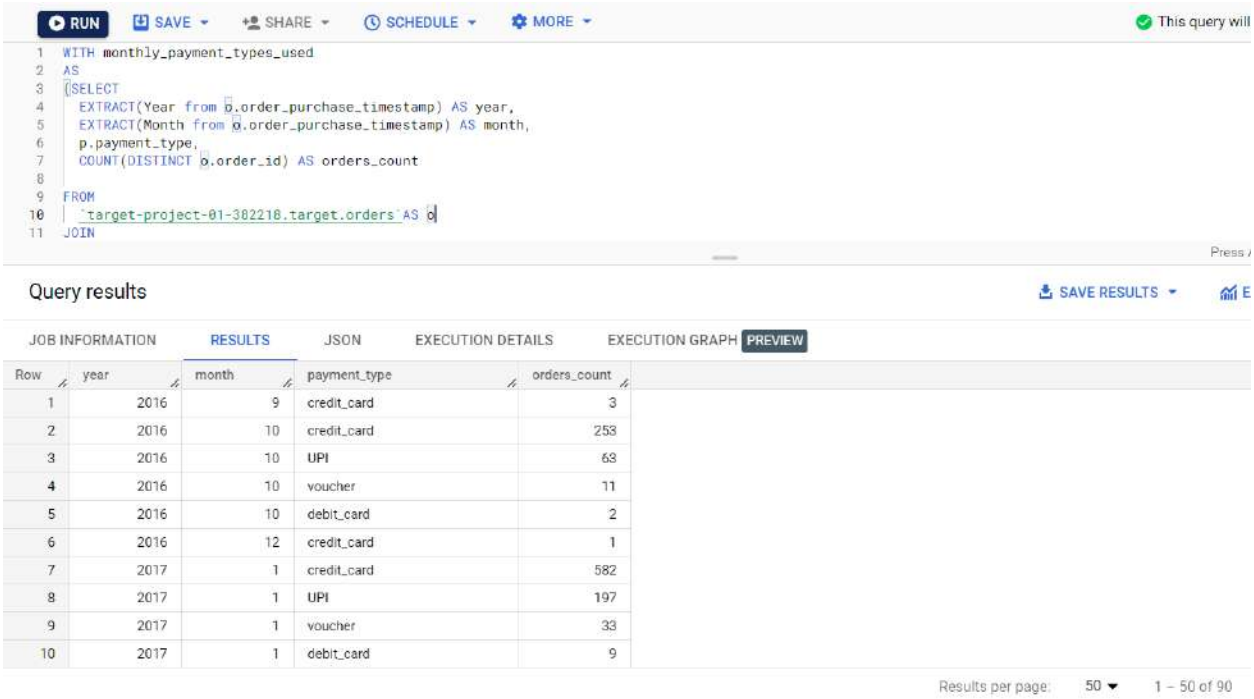
Month over Month count of orders for different payment types

```
WITH monthly_payment_types_used
AS
(SELECT
  o.order_id,
  EXTRACT(Year from o.order_purchase_timestamp) AS year,
  EXTRACT(Month from o.order_purchase_timestamp) AS month,
  p.payment_type

FROM
  `target-project-01-382218.target.orders`AS o
JOIN
  `target-project-01-382218.target.payments`AS p
ON
  o.order_id = p.order_id

)

SELECT
year,
month,
payment_type,
COUNT(DISTINCT order_id) AS orders_count
FROM
  monthly_payment_types_used
GROUP BY
  payment_type,
  year,
  month
ORDER BY
  year,
  month
```



What is the most used payment type in each month?

```
WITH monthly_payment_types_used
AS
(SELECT
  EXTRACT(Year from o.order_purchase_timestamp) AS year,
  EXTRACT(Month from o.order_purchase_timestamp) AS month,
  p.payment_type,
  COUNT(DISTINCT o.order_id) AS orders_count

FROM
  `target-project-01-382218.target.orders`AS o
JOIN
  `target-project-01-382218.target.payments`AS p
ON
  o.order_id = p.order_id
GROUP BY
  p.payment_type,
  EXTRACT(Year from o.order_purchase_timestamp),
  EXTRACT(Month from o.order_purchase_timestamp)
)

SELECT
  year,
  month,
```

```
  payment_type,
  orders_count
FROM
  monthly_payment_types_used
WHERE
  orders_count IN
  (SELECT
    MAX(orders_count)
  FROM
    monthly_payment_types_used
  GROUP BY
    year)
ORDER BY
  year
```

RUN SAVE SHARE SCHEDULE MORE

```
WITH monthly_payment_types_used
AS
(SELECT
  EXTRACT(Year from o.order_purchase_timestamp) AS year,
  EXTRACT(Month from o.order_purchase_timestamp) AS month,
  p.payment_type,
  COUNT(DISTINCT o.order_id) AS orders_count

FROM
  `target-project-01-382218.target.orders`AS o
JOIN
  `target-project-01-382218.target.payments`AS p
ON
  o.order_id = p.order_id
GROUP BY
  p.payment_type, EXTRACT(Year from o.order_purchase_timestamp), EXTRACT(Month from o.order_pur
)

SELECT
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXECUTION GRAPH PREVIEW

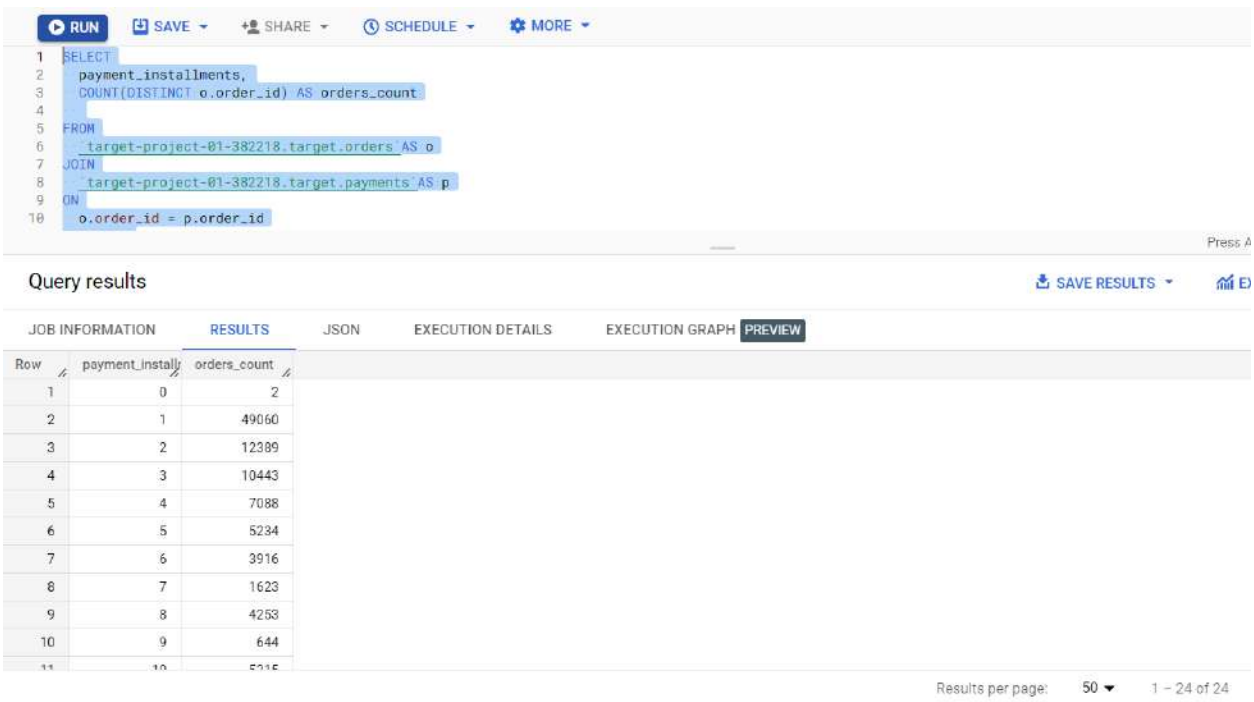
Row	year	month	payment_type	orders_count
1	2016	10	credit_card	253
2	2017	11	credit_card	5867
3	2018	3	credit_card	5674

We can see the **most common payment type used per month in 2016-2018 is <u>credit card</u>**

Count of orders based on the number of payment installments

```
SELECT
  payment_installments,
  COUNT(DISTINCT o.order_id) AS orders_count

FROM
  `target-project-01-382218.target.orders`AS o
JOIN
  `target-project-01-382218.target.payments`AS p
ON
  o.order_id = p.order_id
GROUP BY
  p.payment_installments
ORDER BY
  payment_installments
```



Row	payment_installments	orders_count
1	0	2
2	1	49060
3	2	12389
4	3	10443
5	4	7088
6	5	5234
7	6	3916
8	7	1623
9	8	4253
10	9	644

What is the payment installment option that customers have used for maximum transactions?

```
WITH
orders_per_installment
AS
(
SELECT
  payment_installments,
  COUNT(DISTINCT o.order_id) AS orders_count

FROM
  `target-project-01-382218.target.orders`AS o
JOIN
  `target-project-01-382218.target.payments`AS p
ON
  o.order_id = p.order_id
GROUP BY
  p.payment_installments
ORDER BY
  payment_installments)

SELECT
  *
FROM
  orders_per_installment
ORDER BY
  orders_count DESC
LIMIT 1
```

RUN SAVE SHARE SCHEDULE MORE

```sql
WITH
orders_per_installment
AS
(
SELECT
  payment_installments,
  COUNT(DISTINCT o.order_id) AS orders_count

FROM
  `target-project-01-382218.target.orders`AS o
JOIN
  `target-project-01-382218.target.payments`AS p
ON
  o.order_id = p.order_id
GROUP BY
  p.payment_installments
ORDER BY
```

Query results

JOB INFORMATION | RESULTS | JSON | EXECUTION DETAILS | EXEC

Row	payment_installı	orders_count
1	1	49060

Most orders are made by availing the payment installment **option 1**

Actionable Insights

For more detailed insight refer to each of the “Analysis Question” pages

- Time period of the data is from: **2016 - 2018**
- The customers orders were from **4119 cities and 27 states**
- **Yearly trend of number of orders made**
  - exponential growth of **13609%** seen from **2016 to 2017**
  - growth in the next year **2018** was only **20%**
- **Yearly spread into new cities and states**
  - Orders received from **3111 new cities and 6 new states** in 2017 from 2016
  - No much spread noticed in **2018**
- There is also an observed growth of **20%** from **2017 to 2018** although there has been a reduction in the number of cities from last year
- There is **no seasonality observed** from this limited time period data
- Afternoon time has the **highest number of purchases** made by customers
- Customers from state **SP** has made the **highest orders for Aug - 2018**
- Customers from state **SP** has the maximum number of customers who have ordered in the three years 2016-2018
- Customers from state **RR** has the minimum number of customers who have ordered in the three years 2016-2018
- Increase in the cost of orders is **134% in 2018 from 2017**
- **Maximum orders** between 2016-2018 have been delivered **within 7 days**
- Customer state **AL** has the **fourth highest average time to delivery** and also the **lowest difference to estimated delivery**
- **Most common payment type used per month in 2016-2018 is <u>credit card</u>**

- Most orders are made by availing the payment installment **option 1**

Recommendations

- We can see a high number of active purchases made by customers from **8AM - 11PM** so there is a scope to grow sales by :
  - Implementing different price-discounting strategies for the most common purchased products and product categories.
  - Making the user experience seamless by improving the website and the mobile app. There should be no lag in the payment processing whether it be made through website or the app.
- Although there has been an increase of **134% in the cost of orders** in 2018 from 2017 for months January to August there has been no significant impact on profitability, but we can impose some cost reducing policies (like discount in freight for bulk orders from sellers) to increase the profit percentage for the next year.
- State **AL has the worst delay-to-ontime ratio** among all the states to improve on the delivery of products in this state we need to find what is causing the delay, maybe it is because of inefficient carrier delivery service providers in that state. If that is the case, we need to change to a better carrier to make more ontime delivery to the cities in this state.
- Since the most common mode of payment is **credit card** we can make sure the payment portal for managing such processes are well maintained. We can also provide rewards based on specific cards to the customers.